Software Requirements Specification (SRS)

PROJECT TITLE: Recipe Management System

PROJECT DEVELOPER: Jiashu Hu

PROJECT REPOSITORY (GITHUB LINK): https://github.com/Jiashu-Hu/SE-Project

TABLE OF CONTENT

REVISION HISTORY **3**

1. INTRODUCTION **4**

1.1 Purpose 4
1.2 Document Conventions 4
1.3 Intended Audience and Reading Suggestions 4
1.4 Project Scope 4
1.5 References 4

2. OVERALL DESCRIPTION **4**

2.1 Product Perspective 4
2.2 Product Features 5
2.3 User Classes and Characteristics 5
2.4 Operating Environment 5
2.5 Design and Implementation Constraints 5
2.6 User Documentation 5
2.7 Assumptions and Dependencies 5

3. SYSTEM FEATURES **6**

3.1 User Registration 6
3.2 User Login/Authentication 6

4. EXTERNAL INTERFACE REQUIREMENTS **10**

4.1 User Interfaces 10
4.2 Hardware Interfaces 10
4.3 Software Interfaces 10
4.4 Communications Interfaces 11

5. OTHER NONFUNCTIONAL REQUIREMENTS **11**

5.1 Performance Requirements 11
5.2 Safety Requirements 11
5.3 Security Requirements 11
5.4 Software Quality Attributes 11

6. OTHER REQUIREMENTS **11**

APPENDIX A: GLOSSARY **11**
APPENDIX B: ANALYSIS MODELS **12**
APPENDIX C: ISSUES LIST **15**

Revision History

Name	Date	Reason for Changes	Version
Jiashu Hu	Nov 20	Revise	1.0

1. Introduction

1.1 Purpose

State that this SRS describes the Recipe Management System, version 1.0. Explain that it solves the problem of scattered recipe storage (screenshots, notebooks, online links) by providing a centralized platform for personal recipe organization.

1.2 Document Conventions

- "SHALL" = mandatory requirement
- "SHOULD" = highly desirable requirement
- Priority ratings: High/Medium/Low
- Font conventions (if any)

1.3 Intended Audience and Reading Suggestions

The intended audience includes those interested in developers, testers, instructors, project managers, and users. The customers that should be developed are Section 3 (System Features) and Section 4 (External Interfaces). The focus that testers should have is on Section 3 (Functional Requirements) and Section 5 (Nonfunctional Requirements). The instructors and project managers should go through the whole document at a time with a start to finish reading. All a user needs to do is to look at Section 2 (Overall Description).

1.4 Project Scope

The app allows users to register and in to it. They are able to put, remove, and modify their personal recipes at any time. They are also able to search recipes and then filter them based on either ingredients, time of cooking or eatery. Web interface is terribly simple and very easy to navigate. It saves a lot of time, makes the process of cooking much quicker and more effective, and allows individuals to store all their recipes in an orderly place.

1.5 References

- Project Charter for Recipe Management System (Sep 15, 2025)
- IEEE Std 830-1998 (if applicable)
- Node.js documentation
- Supabase documentation

2. Overall Description

2.1 Product Perspective

The entire system operates on a web based client server platform. Frontend simply consists of HTML, CSS, and JavaScript. Node.js and Express are used as the backends. Database Supabase or, in fact, PostgreSQL. It can be opened in any web browser using any gadget that the user possesses.

2.2 Product Features

The product has user account management, thus users are able to register, log-in and manage their account. They will be able to update, delete, read, and create recipes at any time. When searching recipes, they are able to filter through them by name or category. It allows the users to classify and store all their recipes with ease. The web interface is also reactive and it performs very well on any gadget.

2.3 User Classes and Characteristics

The primary users of this are home cooks, students and other individuals who simply enjoy cooking as a hobby. All they require are bare minimum computer or phone knowledge. They are likely to open it on a daily basis or several times weekly. They primarily make their own recipes, browse, and, in general, scroll through recipes.

2.4 Operating Environment

It is compatible with any up-to-date browser Chrome, Firefox, Safari, Edge, whatever you had. The server is based on Node.js, the database is hosted on Supabase cloud, and the entire setup is launched to some web hosting provider (we will select one later). Of course you require internet, no offline.

2.5 Design and Implementation Constraints

Frontend will be created using HTML, CSS, JavaScript and react. Express and Node.js will be used as the backend. Our choice of database is Supabase. We can hardly spend a lot as our entire budget is just $12 during the whole semester. We also have 13 weeks to complete all. The SDLC approach that we are taking to this project is the Waterfall SDLC. It is web based, and it is not creating any native mobile applications.

2.6 User Documentation

A README file will be placed in the GitHub repository. The help text and tooltips are also inbuilt in the app to guide users on how to use the app. We are developing a quick start guide to new users. We could also create a video tutorial about simple operations but that is not a mandatory thing.

2.7 Assumptions and Dependencies

We are just assuming that all people have a good internet and a modern browser. We will be satisfied with Supabase free tier. GitHub is available as a version control. Whatever free hosting we do choose actually works and does not fall dead at load.

3. System Features

3.1 User Registration

3.1.1 Description and Priority: HIGH - This feature lets new users create accounts

3.1.2 Stimulus/Response Sequences

When a user clicks on Sign Up, a registration form appears on the system. The user enters his/her details and posts it, which in turn the system will create the account and confirm it.

3.1.3 Functional Requirements:

On clicking sign up, the system presents a registration form to the user. The account shall be created by the user and instructions to fill in his/her information and submit it after which the system will create the account and a confirmation is sent (REQ-3.1-1). It will verify whether the format of email is correct or not (REQ-3.1-2). The system will require passwords of at least 8 characters (REQ-3.1-3). The system will also verify the occurrence of the email being in the database (REQ-3.1-4).

3.2 User Login/Authentication

3.2.1 Description and Priority HIGH - This will enable registered users to log in and view their own recipe collection safely. It is very prioritized, as before users can use the app, they are required to authenticate themselves.

3.2.2 Stimulus/Response Sequences

The system presents email and password box in login page, which the user logs in. The user is required to fill his/her information and presses on Login after which the system verifies all the details and allows the user to enter the dashboard. In case the user inputting the credentials in the system does it wrongly, it will display an error message and the user can retry.

3.2.3 Functional Requirements

The system will be able to authenticate an individual based on email and password (REQ-3.2-1). In case the credentials are inaccurate the system will give an error message (REQ-3.2-2). A user session will be created and the system shall take a user to the dashboard when logged in successfully (REQ-3.2-3) and (REQ-3.2-4). The system will also have an option of Forgot Password to the users in case they require it (REQ-3.2-5).

3.3 Add New Recipe

3.3.1 Description and Priority HIGH - It has this feature, which allows the users to create some new recipes and store them in their collection. It has a high priority since cooking ingredients are a fundamental part of the application.

3.3.2 Stimulus/Response Sequences

On the screen, the user clicks on the Add Recipe menu and the system displays a recipe creation form. The user enters the necessary fields and presses the Save button which in

turn makes the system verify the information and save the recipe. In case the user attempts to post an incomplete form the system also points out what fields it considers to be incomplete.

3.3.3 Functional Requirements

The system will also demand a recipe title and it may not be more than 100 characters (REQ-3.3-1). The system will also need an ingredients list (REQ-3.3-2) and what it needs as well is cooking instructions (REQ-3.3-3). Optional categories that the user may choose include category (REQ-3.3-4) and the number of servings (REQ-3.3-5) and prep time in minutes (REQ-3.3-6). The system will maintain the all necessary fields validated prior to saving (REQ-3.3-7) and on the successful save by the system it will display a confirmation to the user (REQ-3.3-8).

3.4 View Recipe Details

3.4.1 Description and Priority

HIGH - This is the feature which displays all the details of a chosen recipe in readable format. It is a priority since its users require to distract their recipes.

3.4.2 Stimulus/Response Sequences

By clicking on a recipe list results, all ingredients of a recipe are extracted and shown by the system. The user can find the title, sources, instructions, category, servings, and the time required to prepare on the page.

3.4.3 Functional Requirements

The system will present the title of the recipe conspicuously (REQ-3.4-1). It will demonstrate the ingredients in a structured list (REQ-3.4-2) and will present the step by step guidelines (REQ-3.4-3). In case category, servings and prep time are present the system shall also show it (REQ-3.4-4). It will also include an offer of an edit and delete button to the owner of the recipe (REQ-3.4-5) and there should be an option of Back to list so that users can go back (REQ-3.4-6).

3.5 Edit Existing Recipe

3.5.1 Description and Priority

MEDIUM - This option allows customers to revise the already saved recipes with new data.

3.5.2 Stimulus/Response Sequences

When the user clicks on the recipe detail page Edit, the system displays a form, which is already pre-filled with the existing information. The user modifies what he/she wishes and clicks "Update" and the system verifies everything and modifies it. In the event that the user decides to click on cancel, the system would revert to recipe detail view and nothing is saved.

3.5.3 Functional Requirements

The system will automatically fill the form with the available recipe information (REQ-

3.5-1). It will be required to validate the changed data prior to making a change (REQ-3.5-2) and it will apply the same validation rules as ones it uses to create a new recipe (REQ-3.5-3). A successful update will result in a confirmation message being displayed by the system (REQ-3.5-4) and it will allow users to cancel the edit without making any change (REQ-3.5-5).

3.6 Delete Recipe

3.6.1 Description and Priority

MEDIUM - This is what allows the users to permanently delete any recipe they do not want under their collection.

3.6.2 Stimulus/Response Sequences

The user gets a confirmation dialog with a popup button that only needs to be clicked on the delete button. In case the user approves the deletion the system will delete the recipe and display a success message. Should the user cancel the system will only close the dialog but nothing will be destroyed.

3.6.3 Functional Requirements

The system will treat the deletion of anything with a confirmation before deleting (REQ-3.6-1). Upon the confirmation made by the user that what the system will do is to permanently delete the recipe in the database (REQ-3.6-2) and that when it will issue a success message (REQ-3.6-3). On deletion the system will take the user back to the recipe list (REQ-3.6-4) and the delete should have a confirmation step (REQ-3.6-5) to avoid accidental deletion.

3.7 Search Recipes by Name

3.7.1 Description and Priority

HIGH - This option enables them to locate particular recipes in the shortest possible time by typing keywords in the title of the recipes. Its priority is high as the users must be provided with a quick method of finding their recipes.

3.7.2 Stimulus/Response Sequences

The user enters a search term on the search box and the system filters the recipes and displays those that are matching it. In the event that the user clears the search the system displays all the recipes once more. Nothing In case nothing talks to the displays a No recipes found message.

3.7.3 Functional Requirements

This system will contain a search input box on the recipe list page (req-3.7-1). It will provide case-insensitive search of the titles of the recipes (REQ-3.7-2) and will display real-time search results as the user type (REQ-3.7-3). In case of no results with the search term the system will show a message stating so (REQ-3.7-4) and it will allow users to empty the search to see all recipes once again (REQ-3.7-5).

3.8 Filter Recipes by Category

3.8.1 Description and Priority MEDIUM -This allows users to see recipes as they are categorised such as Breakfast, dinner or as a Dessert.

3.8.2 Stimulus/Response Sequences
The user has a dropdown where he/she chooses a category and only recipes of that category are displayed. When the user clicks on All Categories the system will display the full list of recipes.

3.8.3 Functional Requirements
The system will have a dropdown filter list of categories at the recipe list page (REQ-3.8-1). It will endorse the predetermined definitions such as Breakfast, Lunch, Dinner, Dessert, Snacks and Other (REQ-3.8-2). The system will show recipes only matching the category of the choice (REQ-3.8-3) and there will be a choice of All Categories where the users have the ability to view the unfiltered list (REQ-3.8-4). The system will also allow the user to use category filter along with search (REQ-3.8-5).

3.9 View Recipe List/Dashboard

3.9.1 Description and Priority HIGH - This feature presents the entire collection of recipes that the user has in a systematic format that he or she can scroll through. It is also high priority as it is the first page the user is greeted with once he is logged in.

3.9.2 Stimulus/Response Sequences
The user enters his or her account and the system shows the dashboard, which contains the recipes list. In case the user has no recipes the system will display a message urging him/her to add the first recipe. The scrollbar allows the user to scroll through the list and the system gives the recipes with the basic information.

3.9.3 Functional Requirements
All the recipes of the user will be shown in the system in the form of a list (REQ-3.9-1). It will display the name of the recipe, type and preparation time of each entry (REQ-3.9-2) and will also have an Add Recipe button on an obvious position (REQ-3.9-3). Every recipe will be clickable in order to see the information (REQ-3.9-4) and recipes will be shown in reverse chronological sequence where the latest ones are placed at the top (REQ-3.9-5). In case the user has no recipes the system will display a helpful message (REQ-3.9-6).

3.10 User Profile Management

3.10.1 Description and Priority LOW - This feature gives users an opportunity to look at and change their account information and preferences.

3.10.2 Stimulus/Response Sequences

When the user clicks on the profile icon, the system will display the profile page which contains the current information of the user. In case the user changes the email or user name, the system will make sure and store the changes. In case the user wants to switch passwords the system asks the user to verify his password first.

3.10.3 Functional Requirements

The system will indicate the email of the user, as well as his or her username (REQ-3.10-1). It will also allow users to update their username (REQ-3.10-2) and it will also allow users to change their password while checking by their current password (REQ-3.10-3). The system will check the validity of new email format before making any changes (REQ-3.10-4) and will have a button which is used to logout on the profile page (REQ-3.10-5). The system will also show the time that the account was opened (REQ-3.10-6).

4. External Interface Requirements

4.1 User Interfaces

The login Page includes an email and password field and a sign up application. The Dashboard displays a list of recipes, search bar, and category filter and an Add Recipe button. The Recipe Gathering contains sections of title, ingredients, direction, category, and other items. Recipe Detail View is the view that shows all the recipe information will have an Edit and Delete button. These will be built in the form of wireframes and mockups.

4.2 Hardware Interfaces

No direct hardware interfaces required. System accessed through standard input devices (keyboard, mouse, touchscreen).

4.3 Software Interfaces

The database is based on Supabase postgreSQL API and it is only supposed to save account of users and recipes. Information is transferred through the API of the REST using the JSON format. The browser APIs also have local storage used to manage the session and the version control and code repository are used by GitHub.

4.4 Communications Interfaces

The protocols employed in the system are HTTP and HTTPS. It adheres to RESTful API design principles of endpoints of a backend and relies on the use of the JSON (data format) both in API requests and responses. The HTTPS traffic passes through port 443.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The amount of time to load the page will be less than 3 seconds and be able to accommodate 100 users at a time. Search results in recipes should appear in less than 2 seconds.

5.2 Safety Requirements

This system will use the automatic backup of data on a daily basis and ensure that no data is lost when users are on.

5.3 Security Requirements

The passwords will be hashed and then stored and the user sessions will have a lifetime of 24 hours. The system will guard the attacks of SQL injection and all communications will be upheld by use of HTTPS.

5.4 Software Quality Attributes

The interface will be user friendly such that in 10 minutes, the typical user can learn how to use it. To achieve reliability the system will have 99% availability. To maintainability the code will be guided by the rules of standard JavaScript and Node.js. In order to be scalable, the database plan will contain 10,000 recipes or more recipes per user.

6. Other Requirements

The database is based on PostgreSQL through Supabase with User, Recipe and Category tables. The project would be open source and define a license such as MIT. The system should be in accordance to basic web accessibility standards that is WCAG 2.0 Level A.

Appendix A: Glossary

- CRUD: Create, Read, Update, Delete operations
- REST API: Representational State Transfer Application Programming Interface
- Supabase: Open-source Firebase alternative
- Responsive Design: Web design approach for optimal viewing across devices

Appendix B: Analysis Models

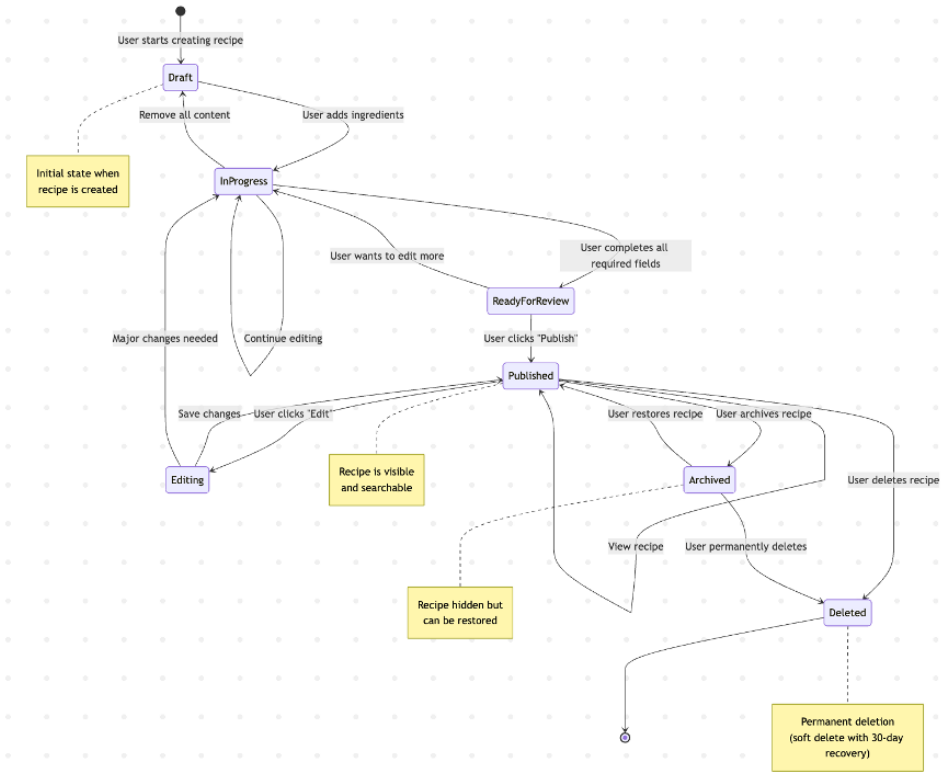

Figure 1: Recipe Lifecycle State Transition Diagram

This state diagram depicts the recipe lifecycle of the system. A user starts building a recipe in the Draft state which is the starting state. Upon the user adding ingredients it will appear on InProgress and may appear back in Draft should the user remove all the content. Once the user finishes all the necessary fields the recipe is transferred to ReadyForReview. The user is able to submit it by clicking on Publish to be Published and at this moment the recipe can be seen and searched. Published The user can also click on Editing to go to Editing state and save to revert to Published, or major changes are required go back to InProgress. There is also the option of archiving a published recipe by the user making it invisible although restorable. In Archived a user is able to reconnect the recipe to Published or discard it. The Deleted state is a soft delete that has 30- days recovery and the user can permanently delete it at the Deleted state to then destroy the lifecycle.

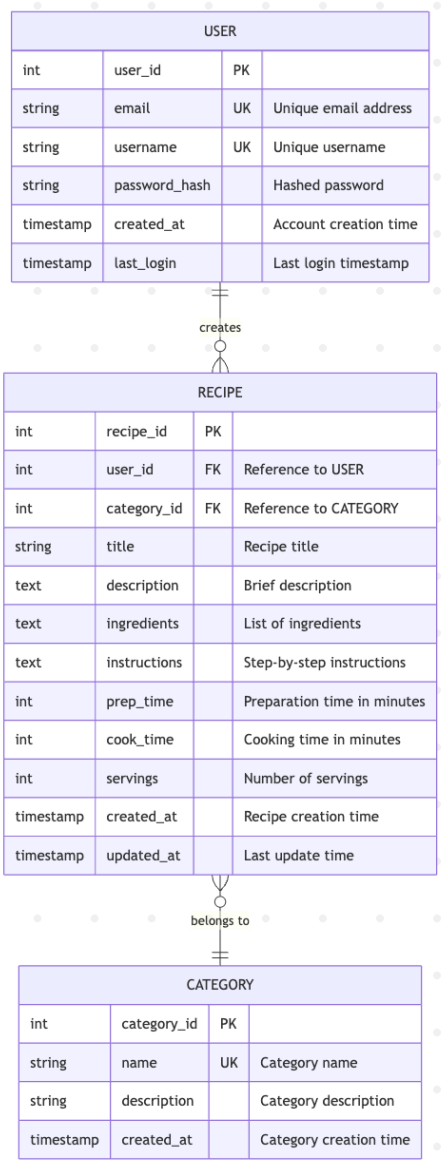

Figure 2: Recipe Management System Database Entity-Relationship Diagram (ERD)

This ER diagram demonstrates the database organizational arrangement of the recipe management system. This consists of three principal tables namely, USER, RECIPE and CATEGORY. user ID is the primary key of USER table, which contains email and username as this table unique keys and password hash, created at, and last login. The RECIPE table has two foreign keys as user_id that references to USER and category_id that references to CATEGORY and has recipeid as primary key. Title, description, ingredients, instructions, prep time, cook time, servings, created at and updated at are also contained in the RECIPE table. The CATEGORY table contains category id as its primary key and name as its unique key and description and created at fields. Relationship displays that a single USER has numerous RECIPEs and a single RECIPE is a part of a single CATEGORY.

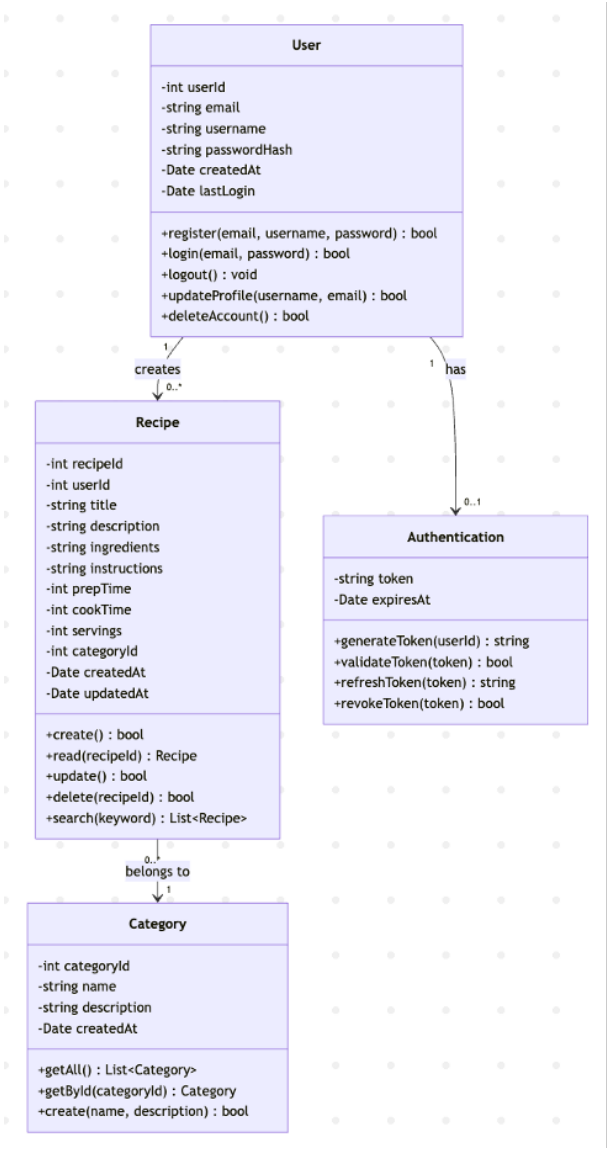

Figure 3: Recipe Management System Object-Oriented Class Diagram

The following class diagram demonstrates the key classes in the recipe management system, and their relationships with one another. The User class contains such attributes as userId, email, username, passwordHash, createdAt, and lastLogin. It has register, logout, deleteaccount, updateprofile and login methods. User class forms a large number of Recipe objects and contains one Authentication object. Recipe class includes such attributes as recipeId, userid, title, description, ingredients, instructions, prepTime, cook time, servings, category-id, created-at, and updated-at. It has create, read, update, delete and search methods. Every Recipe is categorized under a single Category. Attributes of the Category class are categoryId, name, description, and createdAt, and methods of category class to getAll, getById and create. Authentication class deals with tokens as they possess properties of token and expiresAt as well as contains the methods of generateToken, validateToken, refreshToken, and revokeToken.

Appendix C: Issues List

The project will be deployed at vercel